```python
"""D7041-Lab 1
Deborah Aittoklllio debait-2
Joel Willén Joewil-2 """

import numpy as np
from matplotlib import pyplot as plt
from PIL import Image                    # to open/convert/resize images
import os                                # to list files in folders
from statistics import mode
from sklearn.model_selection import KFold
from collections import Counter

from sklearn import datasets
import numpy as np
from matplotlib import pyplot as plt
from sklearn import svm
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, f1_score, accuracy_score
```

Task 1.1

```python
# Setup
root_dir = "../../data/my-dataset"
class_names = ["mug", "book", "keyboard", "shoe", "apple"]
target_size = (28, 28)

# Load images
images = []
labels = []

for class_idx, cname in enumerate(class_names):
    folder = os.path.join(root_dir, cname)
    for fname in os.listdir(folder):
        if fname.lower().endswith((".jpg", ".jpeg", ".png")):
            path = os.path.join(folder, fname)
            img = Image.open(path).convert("RGB")
            img = img.resize((target_size[1], target_size[0]))
            images.append(np.array(img))
            labels.append(class_idx)

# Create arrays
x = np.stack(images, axis=0)
y = np.array(labels)

print("Data shape:", x.shape)
print("y shape:", y.shape, "classes:", len(class_names))
```

```
Data shape: (25, 28, 28, 3)
y shape: (25,) classes: 5
```

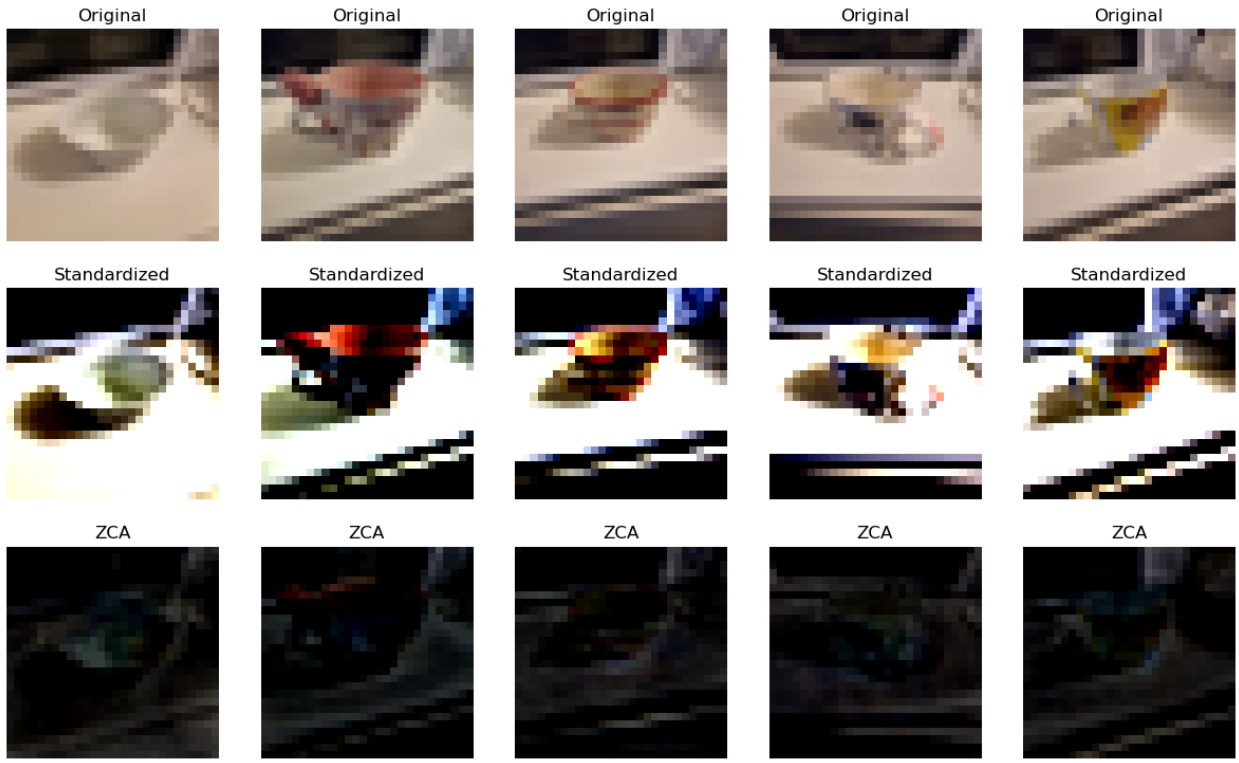
```python
# Show one image per class -  Data Visualization
fig, axes = plt.subplots(1, 5, figsize=(15, 3))
for i, cname in enumerate(class_names):
    idx = list(y).index(i)
    axes[i].imshow(x[idx])
    axes[i].set_title(cname)
    axes[i].axis('off')
plt.show()
```

```python
#Preprocessing


# Normalize
images_norm = x.astype("float32") / 255.0

# Standardize
mean = images_norm.mean(axis=(0,1,2), keepdims=True)
std = images_norm.std(axis=(0,1,2), keepdims=True)
images_std = (images_norm - mean) / (std + 1e-7)

# ZCA Whitening
n, h, w, c = images_std.shape
d = h * w * c
images_flat = images_std.reshape(n, d)

# Center data
mean_img = images_flat.mean(axis=0, keepdims=True)
images_centered = images_flat - mean_img

# Covariance and SVD
cov = np.cov(images_centered, rowvar=False)
U, S, V = np.linalg.svd(cov)

# ZCA transformation
eps = 1e-5
zca_matrix = U @ np.diag(1.0 / np.sqrt(S + eps)) @ U.T
images_zca = images_centered @ zca_matrix.T
images_zca = images_zca.reshape(n, h, w, c)
```

```python
# Show results
fig, axes = plt.subplots(3, 5, figsize=(15, 9))
for i in range(5):
    axes[0, i].imshow(x[i])
    axes[0, i].set_title("Original")
    axes[0, i].axis('off')

    axes[1, i].imshow(images_std[i])
    axes[1, i].set_title("Standardized")
    axes[1, i].axis('off')

    axes[2, i].imshow(images_zca[i])
    axes[2, i].set_title("ZCA")
    axes[2, i].axis('off')

plt.show()
```

```
Clipping input data to the valid range for imshow with RGB data
([0..1] for floats or [0..255] for integers). Got range [-
2.1072924..1.7802889].
Clipping input data to the valid range for imshow with RGB data
([0..1] for floats or [0..255] for integers). Got range [-
0.29401208051285915..0.2757979737612793].
Clipping input data to the valid range for imshow with RGB data
([0..1] for floats or [0..255] for integers). Got range [-
2.1072924..1.9390098].
Clipping input data to the valid range for imshow with RGB data
([0..1] for floats or [0..255] for integers). Got range [-
0.4023756182757155..0.33422337744431685].
Clipping input data to the valid range for imshow with RGB data
([0..1] for floats or [0..255] for integers). Got range [-
2.0557222..1.8599944].
Clipping input data to the valid range for imshow with RGB data
([0..1] for floats or [0..255] for integers). Got range [-
0.3253325097595609..0.3085169995066007].
Clipping input data to the valid range for imshow with RGB data
([0..1] for floats or [0..255] for integers). Got range [-
2.0729122..1.8292363].
Clipping input data to the valid range for imshow with RGB data
([0..1] for floats or [0..255] for integers). Got range [-
0.3453246862427225..0.23922026454696915].
Clipping input data to the valid range for imshow with RGB data
([0..1] for floats or [0..255] for integers). Got range [-
2.0729122..1.919256].
Clipping input data to the valid range for imshow with RGB data
```

```
([0..1] for floats or [0..255] for integers). Got range [-0.3040586011855114..0.3508266292343355].
```




```python
# Simple plot function
def plot_sample(index):
    plt.figure(figsize=(5, 5))
    plt.imshow(x[index])
    plt.title(f"Image {index}: {class_names[y[index]]}")
    plt.axis('off')
    plt.show()

plot_sample(0)
plot_sample(10)
plot_sample(24)
```

Image 0: mug

Image 10: keyboard

Image 24: apple

```
#Summery of the task I have done and what my code does using the
tutorial:
"""
What the code does:

Loads 25 images (5 each of 5 classes)
Creates the required (n,x,y,3) array structure
Shows visualization of original images
Normalizes to [0,1] range
Standardizes (zero-center, unit variance)
Performs ZCA whitening following the tutorial
Shows comparison plots of original then standardized then ZCA
Implements PlotSample function with examples

"""
```

Task 1.2 - Load someone else's dataset

```
# Load data - the file with 11k embeddings
data = np.load('vecs.npy', allow_pickle=True)
dict_data = data.item()                                    #Converted to
dictionary (nested folder structure)
```

```python
# Collect embeddings and labels
embeddings = []
labels = []

for pos in dict_data.keys():
    for digit in dict_data[pos]:
        for embedding in dict_data[pos][digit]:
            labels.append(digit)
            embeddings.append(embedding)

# Convert to arrays
numpy_embed = np.array(embeddings)
numpy_labels = np.array(labels)

# Random shuffle
p = np.random.permutation(len(numpy_embed))
numpy_embed = numpy_embed[p]
numpy_labels = numpy_labels[p]

# Results
print("Task 1.2 done!")
print("Shape:", numpy_embed.shape)
print("Total samples:", len(numpy_embed))
print("Unique labels:", sorted(set(numpy_labels)))
```

```
Task 1.2 done!
Shape: (11302, 1024)
Total samples: 11302
Unique labels: [np.int64(0), np.int64(1), np.int64(2), np.int64(3),
np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8),
np.int64(9)]
```

```python
"""
Task 1.2 Code Summary What i did:

Loads vecs.npy file - Contains multidigit MNIST dataset
Extracts data - Goes through all positions ('1_pos', '2_pos', etc.)
Collects embeddings and labels - For each position and digit, gets all
embeddings
Creates numpy arrays - Converts lists to numpy arrays
Random shuffling - Randomly permutes the order of data and labels
Shows results - Prints shape, total samples, and unique labels


so my understanding so far:
Task 1.1: Remove correlation between images to make it effective for
ML models
Task 1.2: Navigate nested data structure to prepare data for ML
algorithms
```

```
"""
```

Task 2.1

```
# Install Keras directly in Jupyter
!pip install tensorflow keras
```

```
Requirement already satisfied: tensorflow in
/usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: keras in
/usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: absl-py>=1.0.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1
in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!
=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.5)
Requirement already satisfied: requests<3,>=2.21.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (3.2.0)
Requirement already satisfied: typing-extensions>=3.6.6 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard~=2.19.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (3.15.1)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.3)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-
packages (from keras) (13.9.4)
Requirement already satisfied: namex in
/usr/local/lib/python3.12/dist-packages (from keras) (0.1.0)
Requirement already satisfied: optree in
/usr/local/lib/python3.12/dist-packages (from keras) (0.17.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0-
>tensorflow) (0.45.1)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0-
>tensorflow) (2025.10.5)
Requirement already satisfied: markdown>=2.6.8 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0-
>tensorflow) (3.10)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0
in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0-
>tensorflow) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0-
>tensorflow) (3.1.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0-
>rich->keras) (0.1.2)
Requirement already satisfied: MarkupSafe>=2.1.1 in
/usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1-
>tensorboard~=2.19.0->tensorflow) (3.0.3)
```

```python
# Test TensorFlow first
import tensorflow as tf
print("TensorFlow version:", tf.__version__)
```

```
from keras.datasets import mnist
(Xtr, Ltr), (X_test, L_test)=mnist.load_data()

print("Data loaded successfully!")
print("Training shape:", Xtr.shape)
print("Test shape:", X_test.shape)
```

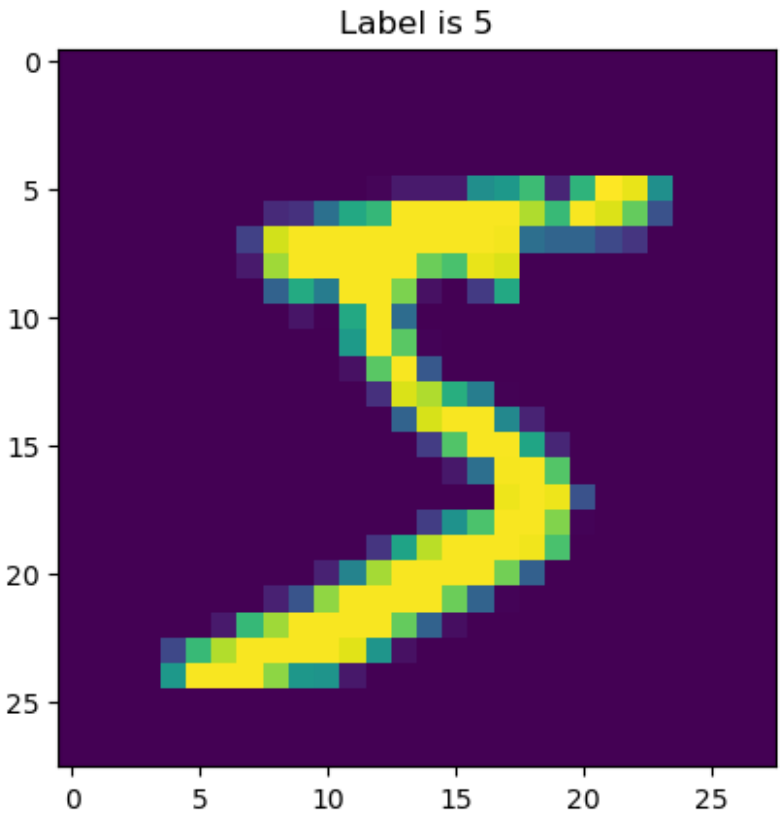
```
TensorFlow version: 2.19.0
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Data loaded successfully!
Training shape: (60000, 28, 28)
Test shape: (10000, 28, 28)
```

1

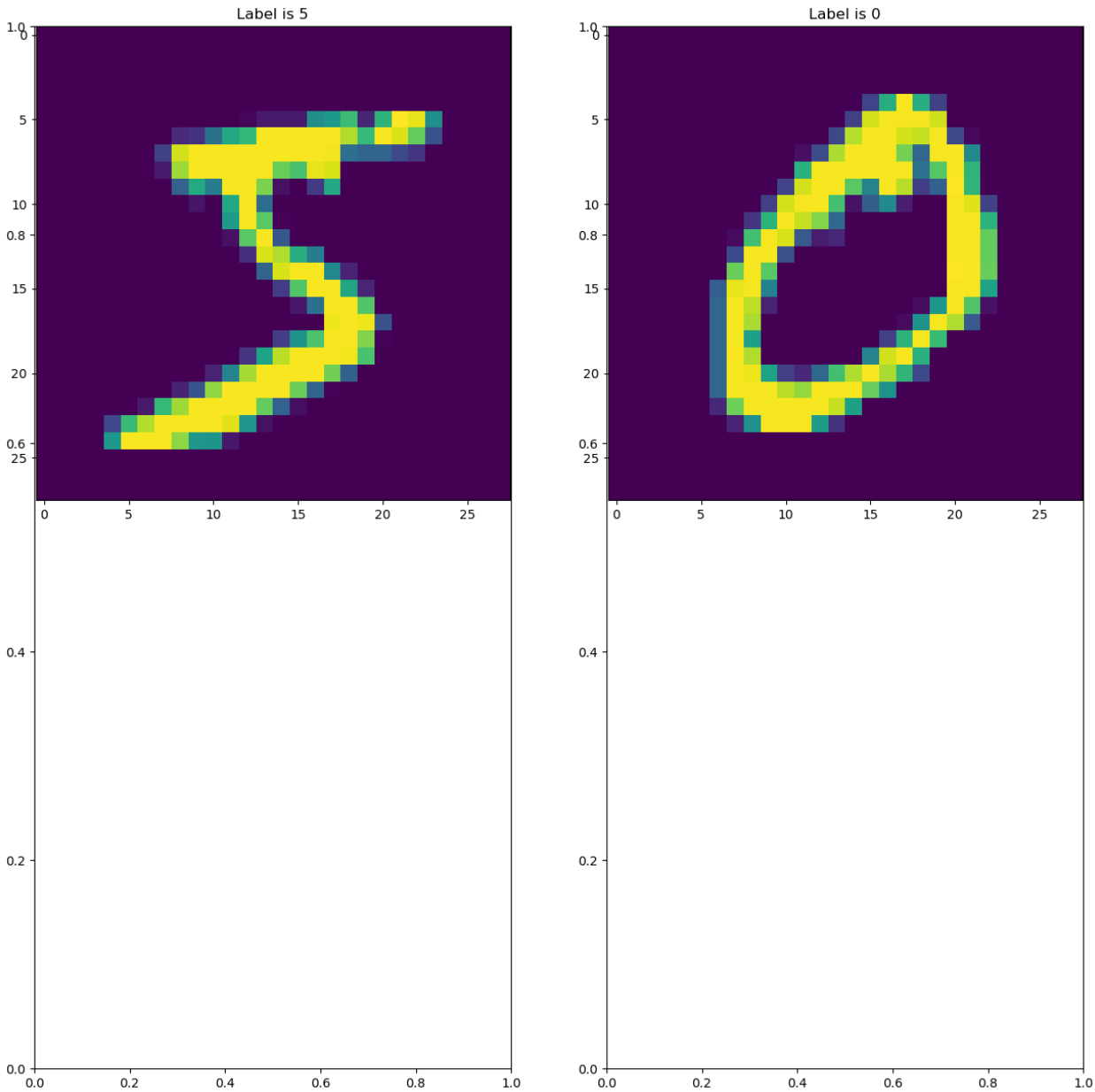
```
Xtr.shape
```

```
(60000, 28, 28)
```

```
Image=Xtr[0,:,:]
Label=Ltr[0]


plt.title('Label is {Label}'.format(Label=Label))
plt.imshow(Image)

plt.show()
plt.close()
```




```
fig, ax=plt.subplots(nrows=1, ncols=2, figsize=(15,15))
ax0=plt.subplot(2,2,1)
ax1=plt.subplot(2,2,2)

Image=Xtr[0,:,:]
Label=Ltr[0]
Image1=Xtr[1,:,:]
Label1=Ltr[1]


ax0.set_title('Label is {Label}'.format(Label=Label))
ax0.imshow(Image)
ax1.set_title('Label is {Label}'.format(Label=Label1))
ax1.imshow(Image1)


plt.show()
plt.close()
```




```
28*28
```

784

```
#Traing phase
num_sample=500
Tr_set=Xtr[:num_sample,:,:]
Ltr_set=Ltr[:num_sample]

# Tr_set=Tr_set.reshape(num_sample,Tr_set.shape[1]*Tr_set.shape[2])

Tr_set=Tr_set.reshape(num_sample,Tr_set.shape[1]*Tr_set.shape[2]).asty
pe(int) #pixels should be integers 0 to 255, bug could be floating
```

```
nummber which could cause calculation errors
Tr_set.shape
```

```
(500, 784)
```

```
"""
Before the change to integers, we can calculate it by hand; Doing so
we get an L1 distance of 1156. Which gives a difference between it and
the python code a difference of 166.
And in the L2 case, by hand we get 469.4. Compared to the python code
it gives a difference of 438,3

"""

img1=Tr_set[100,:]
img2=Tr_set[150,:]
print(img1[180:200])
print(img2[180:200])

pydis1=img1[180:200]
pydis2=img2[180:200]
L1pydis=np.sum(np.abs(pydis1-pydis2))
print(L1pydis)
L2pydis=np.sqrt(np.sum((pydis1-pydis2)**2))
print(L2pydis)
```

```
[  0   2  18  46 136 136 244 255 241 103   0   0   0   0   0   0   0
0
   0   0]
[  0   0   0 139 253 227   8   0   0   0   0   0   0   0   0   0   0
0
   0   0]
1156
469.4230501370805
```

```
def predict(X):
    num_test=X.shape[0]
    Lpred=np.zeros(num_test, dtype=Ltr_set.dtype)

    for i in range(num_test):
        distances=np.sum(np.abs(Tr_set-X[i,:]),axis=1)

        min_index= np.argmin(distances)
        Lpred[i]=Ltr_set[min_index]
    return Lpred

Test_images=X_test.reshape(X_test.shape[0],X_test.shape[1]*
X_test.shape[2])
Labels_predicted=predict(Test_images)

print("Accuracy:", np.mean(Labels_predicted==L_test))
```

```
Accuracy: 0.811
```

```
"""After fixing the bug .astype(int) the accuracy went from
(Accuracy: 0.2649) to (Accuracy: 0.811)"""
```

2 - L2 norm for classification (3. bug fix)

```
def predict_L2(X):
    num_test = X.shape[0]
    Lpred = np.zeros(num_test, dtype=Ltr_set.dtype)

    for i in range(num_test):
        # Original to L2
        distances = np.sqrt(np.sum((Tr_set - X[i,:])**2, axis=1))

        min_index = np.argmin(distances)
        Lpred[i] = Ltr_set[min_index]
    return Lpred


# Test L2 norm
Labels_predicted_L2 = predict_L2(Test_images)
print("L2 norm accuracy:", np.mean(Labels_predicted_L2 == L_test))
```

```
L2 norm accuracy: 0.8294
```

```
"""After fixing the bug .astype(int) the accuracy went from (L2 norm
accuracy: 0.19) to (L2 norm accuracy: 0.8294)"""
```

4 - Extend to k-NN

```
def predict_knn(X, k):
    num_test = X.shape[0]
    Lpred = np.zeros(num_test, dtype=Ltr_set.dtype)

    for i in range(num_test):
        distances = np.sum(np.abs(Tr_set - X[i,:]), axis=1)

        # Find k closest indices (not just 1)
        k_closest_indices = np.argsort(distances)[:k]

        # Get the labels of the k closest images
        k_closest_labels = Ltr_set[k_closest_indices]

        # Vote: find the most common label

        Lpred[i] = mode(k_closest_labels)
```

```
    return Lpred


"""Testing Different k Values"""
# Test images
Test_images = X_test.reshape(X_test.shape[0],
X_test.shape[1]*X_test.shape[2]) # Prepare test data (reshape from 2D
to 1D array), knn needs 1D

# Test run k=1 (original 1-NN)
Labels_predicted_k1 = predict_knn(Test_images, 1)   # finds k nearest
print("k=1 accuracy:", np.mean(Labels_predicted_k1 == L_test))
#Calculate and print accuracy

# Test k=3
Labels_predicted_k3 = predict_knn(Test_images, 3)
print("k=3 accuracy:", np.mean(Labels_predicted_k3 == L_test))

# Test k=5
Labels_predicted_k5 = predict_knn(Test_images, 5)
print("k=5 accuracy:", np.mean(Labels_predicted_k5 == L_test))

# Test k=7
Labels_predicted_k7 = predict_knn(Test_images, 7)
print("k=7 accuracy:", np.mean(Labels_predicted_k7 == L_test))

# Test k=9
Labels_predicted_k9 = predict_knn(Test_images, 9)
print("k=9 accuracy:", np.mean(Labels_predicted_k9 == L_test))
```

```
k=1 accuracy: 0.811
k=3 accuracy: 0.8003
k=5 accuracy: 0.791
k=7 accuracy: 0.7772
k=9 accuracy: 0.7623
```

```
"""In the test the best acc looks to be k= 1"""
```

Task 2.2. Hyperparameters, cross-validation

```
# Randomize and take 500 samples
def shuffle_and_split(X, y, num_samples):
    indices = np.random.permutation(len(X))  # Shuffle all 60,000
    X_shuffled = X[indices]
    y_shuffled = y[indices]
    return X_shuffled[:num_samples], y_shuffled[:num_samples]  # Take
first 500
```

```
num_sample = 500
Tr_set, Ltr_set = shuffle_and_split(Xtr, Ltr, num_sample)  # Our 500
randomized samples
Tr_set = Tr_set.reshape(num_sample, -1).astype(int)

Test_images = X_test.reshape(X_test.shape[0], -1)  # 10,000 test
samples (never shuffled)

# Simple k-NN with L2 norm
def predict_knn_L2(X, k, train_data, train_labels):   # Takes test
data X, finds k nearest neighbors in train_data
    predictions = []
    for test_img in X:
        distances = [np.sqrt(np.sum((train_data[i] - test_img)**2))
for i in range(len(train_data))]
        k_closest_indices = np.argsort(distances)[:k]
        k_closest_labels = train_labels[k_closest_indices]
        predictions.append(mode(k_closest_labels))
    return np.array(predictions)     # Returns predictions for X

# 3fold cross-validation with multiple runs
def find_best_k_multiple_runs(train_data, train_labels, k_values,
n_folds=3, n_runs=20):
    """Find best k by running cross-validation multiple times"""
    best_k_values = []

    print(f"Running {n_runs} cross-validation experiments...")

    for run in range(n_runs):  # Run 20 times with different random
data
        # Randomize data for this run
        indices = np.random.permutation(len(train_data))
        run_data = train_data[indices]
        run_labels = train_labels[indices]

        # Run cross-validation for this run
        kf = KFold(n_splits=n_folds, shuffle=True, random_state=run)
        run_accuracies = []

        for k in k_values:  #  Test each k value (1, 3, 5, 7, 9) for
this run
            fold_accuracies = []

            for train_idx, val_idx in kf.split(run_data):  # Do 3-fold
validation for this specific k
                train_fold = run_data[train_idx] # Split the 500
samples into train/validation for this fold
                train_fold_labels = run_labels[train_idx]
                val_fold = run_data[val_idx]
                val_fold_labels = run_labels[val_idx]
```

```
                predictions = predict_knn_L2(val_fold, k, train_fold,
train_fold_labels) # Test this k on the validation fold
                accuracy = np.mean(predictions == val_fold_labels)
                fold_accuracies.append(accuracy)

            avg_accuracy = np.mean(fold_accuracies)  # Average the 3
fold accuracies for this k
            run_accuracies.append(avg_accuracy)

        # Find best k for this run
        best_k_this_run = k_values[np.argmax(run_accuracies)]
        best_k_values.append(best_k_this_run) #store it

        if run < 5:  # Show first 5 runs
            print(f"Run {run+1}: Best k = {best_k_this_run}")

    # Count frequency of each best k
    k_counts = Counter(best_k_values)

    print(f"\nResults from {n_runs} runs:")
    for k, count in k_counts.most_common():
        print(f"  k={k}: {count} times ({count/n_runs*100:.1f}%)")

    # Find most frequent best k
    best_k = k_counts.most_common(1)[0][0]

    print(f"\nMost frequent best k: {best_k}")
    print(f"This k was best in {k_counts[best_k]} out of {n_runs}
runs")

    return best_k


# Find best k using multiple runs
k_values = [1, 3, 5, 7, 9]
best_k = find_best_k_multiple_runs(Tr_set, Ltr_set, k_values,
n_folds=3, n_runs=20)

# Final test with the best k
final_predictions = predict_knn_L2(Test_images, best_k, Tr_set,
Ltr_set)
final_accuracy = np.mean(final_predictions == L_test)

print(f"\nFinal test accuracy with k={best_k}: {final_accuracy:.4f}")
```

```
Running 20 cross-validation experiments...
Run 1: Best k = 3
Run 2: Best k = 5
Run 3: Best k = 5
```

```
Run 4: Best k = 3
Run 5: Best k = 1

Results from 20 runs:
  k=3: 12 times (60.0%)
  k=5: 4 times (20.0%)
  k=1: 4 times (20.0%)

Most frequent best k: 3
This k was best in 12 out of 20 runs

Final test accuracy with k=3: 0.8451
```

Task 3.1 - Support Vector Machines

```
#Load and Prepare the Iris dataset

iris = datasets.load_iris()
X = iris.data  # 4 features: sepal length, sepal width, petal length,
petal width
y = iris.target # 3 classes: 0=setosa, 1=versicolor, 2=virginica

# Split 80:20 train/test
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=123)

print(f"Training set: {X_train.shape[0]} samples")
print(f"Test set: {X_test.shape[0]} samples")
print(f"Features: {X.shape[1]} (sepal length, sepal width, petal
length, petal width)")
print(f"Classes: {len(np.unique(y))} (setosa, versicolor, virginica)")
```

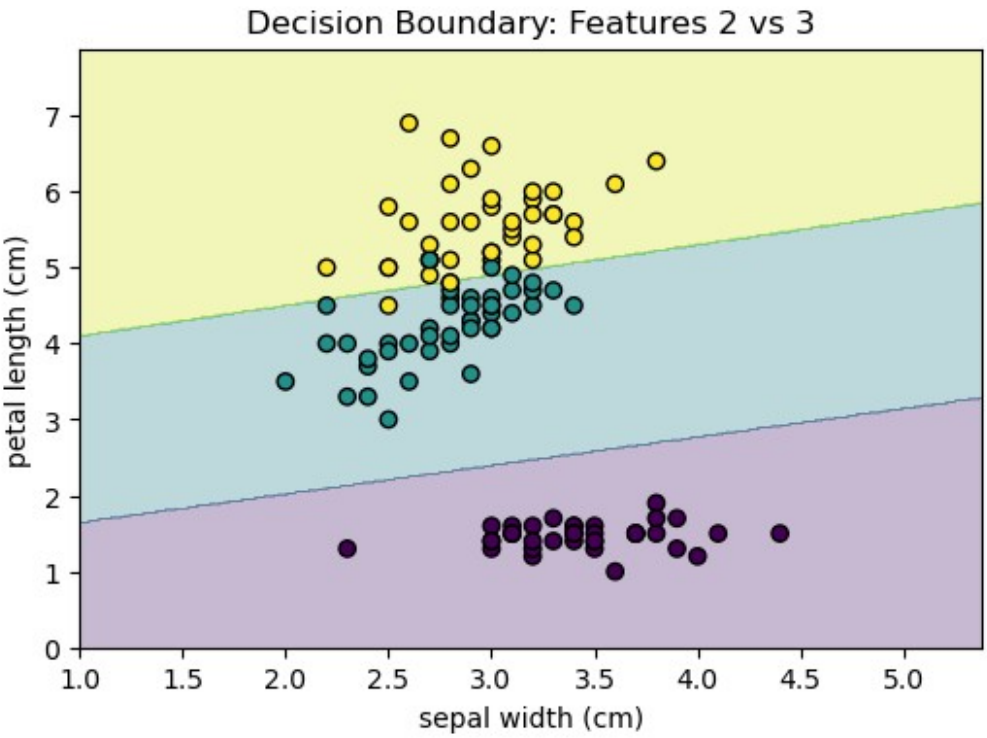
```
Training set: 120 samples
Test set: 30 samples
Features: 4 (sepal length, sepal width, petal length, petal width)
Classes: 3 (setosa, versicolor, virginica)
```

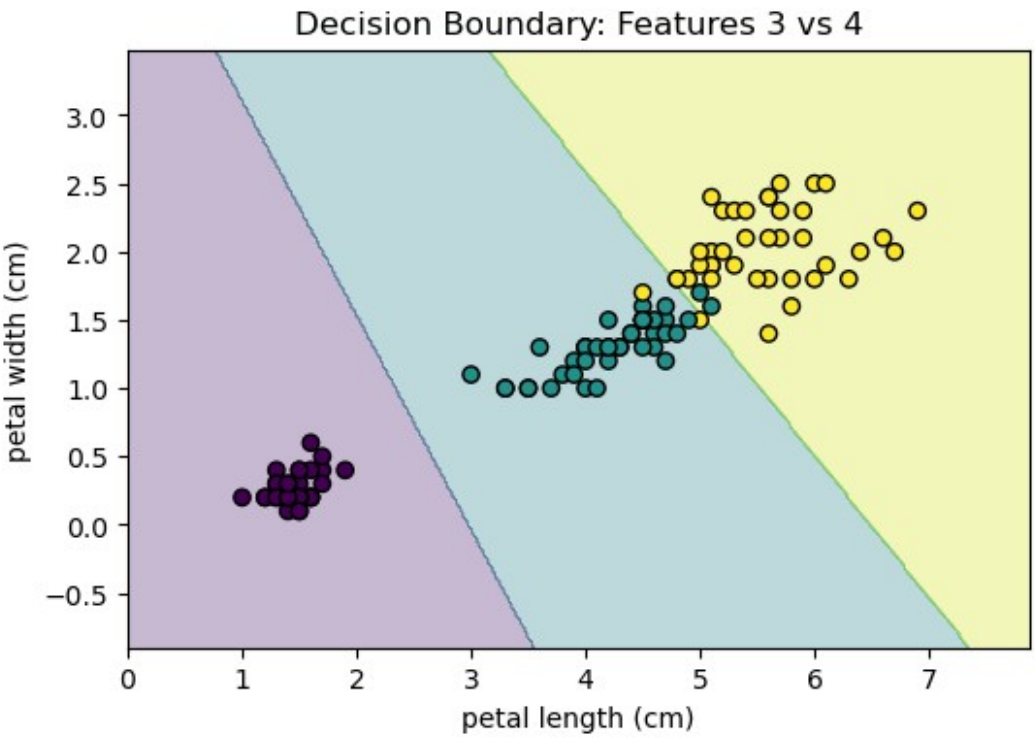
```
"""Train SVM with different kernels (linear, polynomial, RBF)"""



# Create SVM models
linear_ovr = SVC(kernel='linear',
decision_function_shape="ovr").fit(X_train, y_train)
poly_ovr = SVC(kernel='poly',
decision_function_shape="ovr").fit(X_train, y_train)
rbf_ovr = SVC(kernel='rbf',
decision_function_shape="ovr").fit(X_train, y_train)
```

```
linear_ovo = SVC(kernel='linear',
decision_function_shape="ovo").fit(X_train, y_train)
poly_ovo = SVC(kernel='poly',
decision_function_shape="ovo").fit(X_train, y_train)
rbf_ovo = SVC(kernel='rbf',
decision_function_shape="ovo").fit(X_train, y_train)

# Test all models
kernel_texts = ['linear kernel (OVR)', 'polynomial kernel (OVR)', 'RBF
kernel (OVR)',
                'linear kernel(OVO)', 'polynomial kernel (OVO)', 'RBF
kernel (OVO)']

print("Task 3.1: SVM Performance on Iris Dataset")

for i, clf in enumerate((linear_ovr, poly_ovr, rbf_ovr, linear_ovo,
poly_ovo, rbf_ovo)):
    pred = clf.predict(X_test)
    f1 = f1_score(y_test, pred, average='weighted')
    accuracy = accuracy_score(y_test, pred)
    print(kernel_texts[i], "has accuracy:", accuracy, ", F1 score:",
f1)

# Find best kernel
accuracies = []
for clf in (linear_ovr, poly_ovr, rbf_ovr, linear_ovo, poly_ovo,
rbf_ovo):
    pred = clf.predict(X_test)
    accuracy = accuracy_score(y_test, pred)
    accuracies.append(accuracy)

max_acc = max(accuracies)
best_idx = accuracies.index(max_acc)
print("\nBest performing:", kernel_texts[best_idx])
print("Best accuracy:", max_acc)

# Extract support vectors for one-vs-rest (linear kernel)
clf = linear_ovr
print("\nSupport Vectors (One-vs-Rest Linear Kernel)")
print("Total support vectors:", len(clf.support_vectors_))
print("Support vector shape:", clf.support_vectors_.shape)

# Simple decision boundary plot (2D)
def plot_simple_boundary(X_train, y_train, clf, feat1, feat2, title):
    plt.figure(figsize=(6, 4))

    # Use only 2 features for plotting
    X_2d = X_train[:, [feat1, feat2]]
    clf_2d = SVC(kernel='linear',
decision_function_shape="ovr").fit(X_2d, y_train)
```

```
    X1, X2 = np.meshgrid(np.arange(X_2d[:, 0].min() - 1, X_2d[:,
0].max() + 1, 0.02),
                         np.arange(X_2d[:, 1].min() - 1, X_2d[:,
1].max() + 1, 0.02))

    Z = clf_2d.predict(np.c_[X1.ravel(), X2.ravel()])
    Z = Z.reshape(X1.shape)
    plt.contourf(X1, X2, Z, alpha=0.3)
    plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y_train, cmap='viridis',
edgecolors='k')

    plt.xlabel(iris.feature_names[feat1])
    plt.ylabel(iris.feature_names[feat2])
    plt.title(title)
    plt.show()

# Plot decision boundaries
plot_simple_boundary(X_train, y_train, linear_ovr, 1, 2, "Decision
Boundary: Features 2 vs 3")
plot_simple_boundary(X_train, y_train, linear_ovr, 2, 3, "Decision
Boundary: Features 3 vs 4")
```

```
Task 3.1: SVM Performance on Iris Dataset
linear kernel (OVR) has accuracy: 0.9666666666666667 , F1 score:
0.9658761528326745
polynomial kernel (OVR) has accuracy: 0.9666666666666667 , F1 score:
0.9658761528326745
RBF kernel (OVR) has accuracy: 0.9666666666666667 , F1 score:
0.9671550671550672
linear kernel(OVO) has accuracy: 0.9666666666666667 , F1 score:
0.9658761528326745
polynomial kernel (OVO) has accuracy: 0.9666666666666667 , F1 score:
0.9658761528326745
RBF kernel (OVO) has accuracy: 0.9666666666666667 , F1 score:
0.9671550671550672

Best performing: linear kernel (OVR)
Best accuracy: 0.9666666666666667

Support Vectors (One-vs-Rest Linear Kernel)
Total support vectors: 23
Support vector shape: (23, 4)
```


Decision Boundary: Features 2 vs 3
petal length (cm)
sepal width (cm)
0
1
2
3
4
5
6
7
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0


Decision Boundary: Features 3 vs 4
petal width (cm)
petal length (cm)
−0.5
0.0
0.5
1.0
1.5
2.0
2.5
3.0
0
1
2
3
4
5
6
7

Task 4.1-GensimW2V

```
#Initial install of Gensim
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in
/usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in
/usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in
/usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in
/usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1-
>gensim) (2.0.1)
Downloading gensim-4.4.0-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 44.8 MB/s eta
0:00:00

Successfully installed gensim-4.4.0
```

```
"""The Help_functions"""

from scipy import spatial

def creating_text_word8(file1, file2):
    file = open(file1, 'r')
    new_file = open(file2, "w")
    for line in file:
        if line != "\n":
            words = line.split()
            words = trunc_to_eight(words)
            for word in words:
                new_file.write(word + " ")
            new_file.write("\n")

    file.close()
    new_file.close()

def new_tasa(file1, file2):
    maximum = 15000

    dictionary = {}
    text_file = open(file1, "r")
    for line in text_file:
        if line != "\n":
            words = line.split()
```

```
            words = trunc_to_eight(words)
            for word in words:
                if dictionary.get(word) is None:
                    dictionary[word] = 1
                else:
                    dictionary[word] += 1

    text_file.close()

    file = open(file1, 'r')
    new_file = open(file2, "w")
    for line in file:
        if line != "\n":
            words = line.split()
            words = trunc_to_eight(words)
            for word in words:
                if dictionary[word] < maximum:
                    new_file.write(word + " ")
                    # new_file.write("\n")

    file.close()
    new_file.close()


def trunc_to_eight(words):
    i = 0
    for word in words:
        if len(word) > 8:
            words[i] = word[0:8]
        i += 1
    return words

def get_answer_mod(words):
    distance_function = cosine_norm
    min_value = max(distance_function(words[0], words[1]),
distance_function(words[0], words[2]), distance_function(words[0],
words[3]),
                    distance_function(words[0], words[4]))
    i = 1
    while min_value != distance_function(words[0],words[i]):
        i += 1
    if i == 1:
        return 1
    else:
        return 0

def cosine_norm(x, y):
    return 1-spatial.distance.cosine(x,y)
```

```
"""The actual GensimW2V. Where we alterante the dimensionality to then
later compare the accuracies between the different values."""

# import modules & set up logging
import gensim, logging, numpy as np
import help_functions as hf
import nltk

#@author: The first version of this code is the courtesy of Vadim
Selyanik


logging.basicConfig(format='%(asctime)s : %(levelname)s : %
(message)s', level=logging.INFO)
nltk.download('wordnet')
lemmatizer = nltk.WordNetLemmatizer() # create a lemmatizer

sentences = []
file = open("lemmatized.text", "r")

for line in file: # read the file and create list which contains all
sentences found in the text
    sentences.append(line.split())
# train word2vec on the two sentences

dimension = 500 # parameter for Word2vec size of vectors for word
embedding

threshold = 0.00055 # parameter for Word2vec


sum = 0.0

#
model = gensim.models.Word2Vec(sentences, min_count=1,
sample=threshold, sg=1,vector_size=dimension) # create model using
Word2Ve with the given parameters
words = list(model.wv.key_to_index.keys())
#
print(len(words)) # check the length of the vocabulary which was
formed by Word2Vec

#The rest implements passing TOEFL tests
i = 0 #counter for TOEFL tests
number_of_tests = 80
text_file = open('new_toefl.txt', 'r')
right_answers = 0 # variable for correct answers
number_skipped_tests = 0 # some tests could be skipped if there are no
corresponding words in the vocabulary extracted from the training
corpus
while i < number_of_tests:
```

```
            line = text_file.readline() #read line in the file
            words = line.split() # extract words from the line
            try:
                words =
[lemmatizer.lemmatize(lemmatizer.lemmatize(lemmatizer.lemmatize(word,
'v'), 'n'), 'a') for word in
                         words] # lemmatize words in the current test
                vectors = []
                if words[0] in words: # check if there embedding for
the query word
                    k = 1 #counter for loop iterating over 5 words in
the test
                    vectors.append(model.wv[words[0]])
                    while k < 5:
                        if words[k] in words: # if alternative has the
embedding
                            vectors.append(model.wv[words[k]]) #assing
the learned vector
                        else:
                            vectors.append(np.random.randn(dimension))
#assing random vector
                        k += 1
                    right_answers += get_answer_mod(vectors) #find the
closest vector and check if it is the correct answer

            except KeyError: # if there is no representation for the
query vector than skip
                number_skipped_tests += 1
                print("skipped test: " + str(i) + "; Line: " +
str(words))
            except IndexError:
                print(i)
                print(line)
                print(words)
                break
            i += 1
text_file.close()
sum += 100 * float(right_answers) / float(number_of_tests) #get the
percentage
print("Threshold ferq = "+ str(threshold)+" Percentage of correct
answers: " + str(sum) + "%")
#%%
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

```
70583
skipped test: 10; Line: ['salute', 'greeting', 'information',
'ceremony', 'privilege']
skipped test: 32; Line: ['distribute', 'circulate', 'commercialize',
```

```
'research', 'acknowledge']
skipped test: 43; Line: ['verbally', 'orally', 'overtly', 'fittingly',
'verbosely']
skipped test: 53; Line: ['halfheartedly', 'apathetically',
'customarily', 'bipartisanly', 'unconventionally']
skipped test: 60; Line: ['generally', 'broadly', 'descriptively',
'controversially', 'accurately']
skipped test: 69; Line: ['expeditiously', 'rapidly', 'frequently',
'actually', 'repeatedly']
Threshold ferq = 0.00055 Percentage of correct answers: 66.25%

"""
Full results:

Dimensions = 50:

70583
skipped test: 10; Line: ['salute', 'greeting', 'information',
'ceremony', 'privilege']
skipped test: 32; Line: ['distribute', 'circulate', 'commercialize',
'research', 'acknowledge']
skipped test: 43; Line: ['verbally', 'orally', 'overtly', 'fittingly',
'verbosely']
skipped test: 53; Line: ['halfheartedly', 'apathetically',
'customarily', 'bipartisanly', 'unconventionally']
skipped test: 60; Line: ['generally', 'broadly', 'descriptively',
'controversially', 'accurately']
skipped test: 69; Line: ['expeditiously', 'rapidly', 'frequently',
'actually', 'repeatedly']
Threshold ferq = 0.00055 Percentage of correct answers: 65.0%

dimensions = 500:

70583
skipped test: 10; Line: ['salute', 'greeting', 'information',
'ceremony', 'privilege']
skipped test: 32; Line: ['distribute', 'circulate', 'commercialize',
'research', 'acknowledge']
skipped test: 43; Line: ['verbally', 'orally', 'overtly', 'fittingly',
'verbosely']
skipped test: 53; Line: ['halfheartedly', 'apathetically',
'customarily', 'bipartisanly', 'unconventionally']
skipped test: 60; Line: ['generally', 'broadly', 'descriptively',
'controversially', 'accurately']
skipped test: 69; Line: ['expeditiously', 'rapidly', 'frequently',
'actually', 'repeatedly']
Threshold ferq = 0.00055 Percentage of correct answers: 66.25%

Dimensions = 5000
70583
```

```
skipped test: 10; Line: ['salute', 'greeting', 'information',
'ceremony', 'privilege']
skipped test: 32; Line: ['distribute', 'circulate', 'commercialize',
'research', 'acknowledge']
skipped test: 43; Line: ['verbally', 'orally', 'overtly', 'fittingly',
'verbosely']
skipped test: 53; Line: ['halfheartedly', 'apathetically',
'customarily', 'bipartisanly', 'unconventionally']
skipped test: 60; Line: ['generally', 'broadly', 'descriptively',
'controversially', 'accurately']
skipped test: 69; Line: ['expeditiously', 'rapidly', 'frequently',
'actually', 'repeatedly']
Threshold ferq = 0.00055 Percentage of correct answers: 65.0%


"""

"""

Due to computational struggles several runs with larger dimensions
could not be completed to get a good mean accuracy.
So we end up with a accuracy of 65% on 5000 dimensions, taking over an
hour to run. 500 dimensions gets an accuracy of 66.25% and lastly 50
dimensions gets and accuracy of 65%.

Now, if we we're able to do this multiple times and get a better mean
average. The most likely outcome would be an increase in accuracy as
dimensionality rises  to an extent. W2V makes use of dimensions to
increase its ability to learn more semantic patterns as the dimensions
are a representation of the learned features.
W2V learns the vectors unlike RI which means that increasing
dimensionality improves its learning but it will overfit more and the
features will become more redundant more as the dimensions get really
large.


"""
```

Task 4.2-Random Indexing

```
"""First the text_functions"""

import numpy as np
from scipy import spatial
#import scipy


def get_random_word_vector(dimension, k):
    positions = np.random.permutation(dimension)[0:2 * k]
```

```
    minus_positions = positions[0:k]
    plus_positions = positions[k:2 * k]
    v = np.array(np.zeros(dimension),np.int8)
    v[minus_positions] = -1
    v[plus_positions] = 1
    return v

def get_random_word_vector_bin(dimension):
    k = (int)(dimension/2)
    positions = np.random.permutation(dimension)[0:2 * k]
    minus_positions = positions[0:k]
    plus_positions = positions[k:2 * k]
    v = np.array(np.zeros(dimension), np.int8)
    v[minus_positions] = -1
    v[plus_positions] = 1
    return v

def get_answer(letter, word,  values):
    answer_list = ['A', 'B', 'C', 'D']
    distance_function = cosine_norm
    min_value = max(distance_function(word, values[0]),
distance_function(word, values[1]), distance_function(word,
values[2]),
                    distance_function(word, values[3]))
    i = 0
    while min_value != distance_function(word,values[i]):
        i += 1
    if answer_list[i] == letter:
        return 1
    else:
        return 0

def get_answer_mod(words):
    distance_function = cosine_norm
    min_value = max(distance_function(words[0], words[1]),
distance_function(words[0], words[2]), distance_function(words[0],
words[3]),
                    distance_function(words[0], words[4]))
    i = 1
    # while i <=4 and min_value !=
distance_function(words[0],words[i]):
    #     i+=1
    if min_value == distance_function(words[0],words[1]):
        return 1
    else:
        return 0

def euclidian_norm(x,y):
    return np.linalg.norm(x-y)
```

```python
def cosine_norm(x, y):
    return 1-spatial.distance.cosine(x,y)

def trunc_to_eight(words):
    i = 0
    for word in words:
        if len(word) > 8:
            words[i] = word[0:8]
        i += 1
    return words

"""

The dirst part of this task is to identify an issue. The RI code we
have been given is only able to handle a window size of 2. Which is
able to do our task but we need to find why and what is the reason for
this lack of ability to handle other window sizes.

The part of the code that will struggle with this is the part of the
logic where it calls for if len(lines[X].....) this comes up as an
issue. As this logic is only viable when there is a window_size==2.
And we also create a very specific lines definition with lines=[[],[],
[],[]].
The code is unable to generalize the amount of neighbours when the
window size changes. This can be fixed or worked around by making the
lines more generalized and less hyperspecific on there being a certain
amount of neighboues for each index of the lines[i].


"""

import numpy as np
import text_functions as tf
import nltk

#@author: The first version of this code is the courtesy of Vadim
Selyanik

threshold = 15000 # Frequency threshold in the corpus ??
dimension = 10000 # Dimensionality for high-dimensional vectors
lemmatizer = nltk.WordNetLemmatizer()  # create an instance of
lemmatizer
ones_number = 2 # number of nonzero elements in randomly generated
high-dimensional vectors
window_size = 2 #number of neighboring words to consider both back and
forth. In other words number of words before/after current word
zero_vector = np.zeros(dimension)
test_name = "new_toefl.txt" # file with TOEFL dataset
data_file_name = "lemmatized.text" # file with the text corpus
```

```
amount_dictionary = {}

# Count how many times each word appears in the corpus
text_file = open(data_file_name, "r")
for line in text_file:
    if line != "\n":
        words = line.split()
        for word in words:
            if amount_dictionary.get(word) is None:
                amount_dictionary[word] = 1
            else:
                amount_dictionary[word] += 1
text_file.close()

dictionary = {} #vocabulary and corresponing random high-dimensional
vectors
word_space = {} #embedings




#Create a dictionary with the assigned random high-dimensional vectors
text_file = open(data_file_name, "r")
for line in text_file: #read line in the file
    words = line.split() # extract words from the line
    for word in words:  # for each word
        if dictionary.get(word) is None: # If the word was not yed
added to the vocabulary
            if amount_dictionary[word] < threshold:
                dictionary[word] =
tf.get_random_word_vector(dimension, ones_number) # assign a
            else:
                dictionary[word] = np.zeros(dimension) # frequent
words are assigned with empty vectors. In a way they will not
contribute to the word embedding

text_file.close()


#Note that in order to save time we only create embeddings for the
words needed in the TOEFL task

    #Find all unique words amongst TOEFL tasks and initialize their
embeddings to zeros
number_of_tests = 0
text_file = open(test_name, "r") #open TOEFL tasks
for line in text_file:
        words = line.split()
        words =
[lemmatizer.lemmatize(lemmatizer.lemmatize(lemmatizer.lemmatize(word,
```

```
'v'), 'n'), 'a') for word in
                  words] # lemmatize words in the current test
        word_space[words[0]] = np.zeros(dimension)
        word_space[words[1]] = np.zeros(dimension)
        word_space[words[2]] = np.zeros(dimension)
        word_space[words[3]] = np.zeros(dimension)
        word_space[words[4]] = np.zeros(dimension)
        number_of_tests += 1
text_file.close()


    # Processing the text to build the embeddings
text_file = open(data_file_name, "r")
lines = [[],[],[],[]] # neighboring lines
i = 2
while i < 4:
        line = "\n"
        while line == "\n":
            line = text_file.readline()
        lines[i] = line.split()
        i += 1

line = text_file.readline()
while line != "":
        if line != "\n":
            lines.append(line.split())
            words = lines[2]
            length = len(words)
            i = 0
            while i < length:
                if not (word_space.get(words[i]) is None):
                    k = 1
                    word_space_vector = word_space[words[i]]
                    while (i - k >= 0) and (k <= window_size):
#process left neighbors of the focus word
                        word_space[words[i]] =
np.add(word_space[words[i]], np.roll(dictionary[words[i - k]], -1))
                        k += 1
                    # Handle different situations if there was not
enough neighbors on the left in the current line
                    if k <= window_size and (len(lines[1])>0):
                        if len(lines[1]) < 2:
                            if k != 1: #if one word on the left was
already added
                                word_space[words[i]] =
np.add(word_space[words[i]], np.roll(dictionary[lines[1][0]], -1))
#update word embedding
                            else:
                                word_space[words[i]] =
np.add(word_space[words[i]],
```

```
np.roll(dictionary[lines[1][0]], -1)) #update word embedding
                                word_space[words[i]] =
np.add(word_space[words[i]],

np.roll(dictionary[lines[0][len(lines[0]) - 1]], -1)) #update word
embedding
                        else:
                            if k != 1:
                                word_space[words[i]] =
np.add(word_space[words[i]],

np.roll(dictionary[lines[1][len(lines[1]) - 1]], -1)) #update word
embedding
                            else:
                                word_space[words[i]] =
np.add(word_space[words[i]],

np.roll(dictionary[lines[1][len(lines[1]) - 1]], -1)) #update word
embedding
                                word_space[words[i]] =
np.add(word_space[words[i]],

np.roll(dictionary[lines[1][len(lines[1]) - 2]], -1)) #update word
embedding

                    k = 1
                    while (i + k < length) and (k <= window_size):
#process right neighbors of the focus word
                        word_space[words[i]] =
np.add(word_space[words[i]], np.roll(dictionary[words[i + k]], 1))
#update word embedding
                        k += 1
                    if k <= window_size:
                        if len(lines[3]) < 2:
                            if k != 1:
                                word_space[words[i]] =
np.add(word_space[words[i]], np.roll(dictionary[lines[3][0]], 1))
#update word embedding
                            else:
                                word_space[words[i]] =
np.add(word_space[words[i]], np.roll(dictionary[lines[3][0]], 1))
#update word embedding
                                word_space[words[i]] =
np.add(word_space[words[i]], np.roll(dictionary[lines[4][0]], 1))
#update word embedding
                        else:
                            if k != 1:
                                word_space[words[i]] =
np.add(word_space[words[i]], np.roll(dictionary[lines[3][0]], 1))
```

```
#update word embedding
                            else:
                                word_space[words[i]] =
np.add(word_space[words[i]], np.roll(dictionary[lines[3][0]], 1))
#update word embedding
                                word_space[words[i]] =
np.add(word_space[words[i]],

np.roll(dictionary[lines[3][1]], 1))

                i += 1
            lines.pop(0)
        line = text_file.readline()



#Testing of the embeddings on TOEFL
a = 0.0 # accuracy of the encodings
i = 0
text_file = open(test_name, 'r')
right_answers = 0.0 # variable for correct answers
number_skipped_tests = 0.0 # some tests could be skipped if there are
no corresponding words in the vocabulary extracted from the training
corpus
while i < number_of_tests:
        line = text_file.readline() #read line in the file
        words = line.split()  # extract words from the line
        words =
[lemmatizer.lemmatize(lemmatizer.lemmatize(lemmatizer.lemmatize(word,
'v'), 'n'), 'a') for word in
                  words]  # lemmatize words in the current test
        try:

            if not(amount_dictionary.get(words[0]) is None): # check
if there word in the corpus for the query word
                k = 1
                while k < 5:
                    # if amount_dictionary.get(words[k]) is None:
                    #     word_space[words[k]] =
np.random.randn(dimension)
                    if np.array_equal(word_space[words[k]],
zero_vector): # if no representation was learnt assign a random vector
                        word_space[words[k]] =
np.random.randn(dimension)
                    k += 1
                right_answers +=
tf.get_answer_mod([word_space[words[0]],word_space[words[1]],word_spac
e[words[2]],

word_space[words[3]],word_space[words[4]]]) #check if word is
```

```
predicted right
        except KeyError: # if there is no representation for the query
vector than skip
            number_skipped_tests += 1
            print("skipped test: " + str(i) + "; Line: " + str(words))
        except IndexError:
            print(i)
            print(line)
            print(words)
            break
        i += 1
text_file.close()
a += 100 * right_answers / number_of_tests
print(str(dimension) + " Percentage of correct answers: " + str(100 *
right_answers / number_of_tests) + "%")
```

```
10000 Percentage of correct answers: 70.0%
```

```
"""
1000 dimensions: First run: 70.0%. Second run: 67.5%. Third run:
70.0%. Fourth run: 67.5%. Fifth run: 68.75%. Mean accuracy 68,75%

5000 dimensions: First run: 72.5%. Second run: 68.75%. Third run:
72.5%. Fourth run: 70.0%. Fifth run: 73.75%. Mean Accuracy: 71,5%

10000 dimensions: First run: 75.0%. Second run: 76,25   Third run:
75.0%. Fourth run: 76.25%. Fifth run: 70.0%. Mean accuracy: 74,5%

"""

"""

So we see that the mean accuracy of the RI increases with the
dimensionality.

RI's accuracy is improved as its dimensionality increases. This is
becuase the higher the dimensionality the better the random vectors in
RI are at avoiding collisions. So increasing dimensionality reduces
noise through reducing these collisions which leads to higher
accuracy.


"""

"""Conclusion"""

"""
Firstly we can clealy see both models perform above picking random
options. With W2V achiving a highest accuracy of 66,25% and RI
```

```
achieving a mean accuracy of 74,5%. In this case RI came out with the
highest accuracy and it acted as expected with improved performence
with increased dimensionality.
W2V did also act as expected with increased dimensionality as it
improves with a higher dimensionality but this improvement does not
scale like RI and instead plataues with overfitting becoming a more
prevelant issue.
W2V also suffered at the cost of its very large computational demand.
The need to train makes it scale fast with the increase in
dimensionality and become very computationally expensive.
Ri instead stays relativly cheap as it creates its vectors in an
instant and doesn't require the training that W2V needs, relying on
vector addition instead.
So in conclusion RI managed to outperform W2V slightly in this
synonyms task in terms of accuracy and more so computational cost.

"""
```